

VIT Bhopal University

Real-Time Object Detection Using YOLO11

Computer Vision – Final Project Report

Submitted by

Abhijai Raghuvanshi

Course / Subject

Computer Vision

Faculty

Dr Vikas Pal

Academic Year

2026 – 2027

Submission Date

18 September 2026

Abstract

This project is a simple real-time object detection application that works with a webcam. The live camera frames are read using OpenCV and passed to a pretrained YOLO11n model. The detected objects are shown on the video with bounding boxes, class names and confidence values. The application also displays basic statistics such as FPS and the number of detected objects. During the run, the user can save a snapshot, clear the statistics, change the confidence threshold, or close the program. A short session summary is saved as a JSON file when the program ends. The project focuses on building and testing the application around the pretrained model rather than training a new model.

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Scope
5. Functional and Non-Functional Requirements
6. Tools and Technologies
7. Dataset and Model
8. System Design
9. Workflow
10. Design Diagrams
11. Implementation
12. User Controls
13. Testing
14. Results
15. Screenshots and Evidence
16. Limitations
17. Challenges and Learning Outcomes
18. Future Enhancements
19. Conclusion
20. References

1. Introduction

1.1 Computer Vision

Computer Vision is used to make computers understand information from images and videos. Common tasks include image classification, object detection, image segmentation and object tracking.

1.2 Object Detection

Object detection identifies objects in an image and also finds their locations. The output is usually a bounding box, a class name and a confidence value. For example, a frame may contain a person and a laptop, and the system can show both objects separately.

1.3 Real-Time Detection

In this project the input is a continuous webcam stream instead of a single image. This makes speed important because the system has to capture, process and display frames continuously. A smaller model is therefore useful for keeping the application responsive.

2. Problem Statement

A normal webcam only provides a video feed and does not tell the user what objects are present in the scene. The aim of this project is to build a program that can take the live webcam feed, detect common objects automatically, show the results on the screen and provide a few useful live statistics.

3. Objectives

The main objectives of the project are:

- Capture live video from a webcam using OpenCV.
- Run object detection on each frame using pretrained YOLO11n.
- Display bounding boxes, object names and confidence values.
- Show FPS and object-count information while the program is running.
- Provide simple keyboard controls for the user.
- Save an annotated snapshot and a session summary.
- Keep the code divided into separate modules and test the important parts.

4. Scope

4.1 Included

The project includes one webcam, real-time YOLO11n detection, the 80 COCO object classes supported by the pretrained model, live bounding boxes and labels, FPS and object counts, keyboard controls, snapshot saving and a JSON session summary.

4.2 Not Included

The project does not include custom model training, fine-tuning, object tracking, multiple cameras, image uploading, cloud deployment, a web dashboard, distance estimation or voice control. These features were kept outside the scope so the main application could be completed and tested properly.

5. Functional and Non-Functional Requirements

5.1 Functional Requirements

- FR1 – Webcam Capture: open the selected camera and read frames.
- FR2 – Object Detection: run YOLO11n on each frame and return detected objects.
- FR3 – Visualisation: draw boxes, labels and confidence values on the frame.
- FR4 – Live Analytics: show FPS, current object count and class counts.
- FR5 – User Controls: respond to Q/ESC, S, C and +/- keys.
- FR6 – Snapshot: save the current annotated frame as a JPEG.
- FR7 – Session Summary: save the session statistics as JSON when the program exits.

5.2 Non-Functional Requirements

- Performance: the program should feel responsive enough for live use.
- Usability: it should start with one command and show the controls on screen.
- Reliability: camera and model errors should be reported clearly and resources should be released on exit.
- Maintainability: the code should be divided into modules with clear responsibilities.
- Portability: the application should work on common desktop platforms with the appropriate OpenCV camera backend.
- Testability: important application logic should be testable without requiring a live camera.

6. Tools and Technologies

Python is used for the application. OpenCV handles webcam capture, drawing and the display window. Ultralytics provides the YOLO11 API and pretrained weights. YOLO11n is the model used for detection. PyTorch performs the underlying model computation, NumPy is used for array data, and pytest is used for unit testing. Git and GitHub are used for version control and project submission.

Main technologies:

- Python 3.10+
- OpenCV (opencv-python)
- Ultralytics YOLO11
- YOLO11n pretrained weights
- PyTorch
- NumPy
- pytest
- Git / GitHub

7. Dataset and Model

7.1 YOLO11n

The application uses the pretrained yolo11n.pt model provided through Ultralytics. The model is loaded at the start of the application and is not trained or fine-tuned as part of this project.

7.2 COCO Classes

The pretrained model is based on the COCO object classes and can report 80 common classes such as person, car, bicycle, dog, cat, bottle, chair and laptop. Because the project uses the pretrained model directly, detection is limited to the classes supported by that model.

7.3 Why YOLO11n?

The nano version was selected mainly because this is a real-time webcam project. It is smaller than the larger YOLO11 variants and is easier to run on a normal laptop. The goal was to get a practical live application rather than spend the project time on model training.

8. System Design

8.1 Overall Architecture

The program is organised around a main loop. The main module gets a frame from the camera, sends it to the detector, updates the statistics, draws the results and displays the frame. Keyboard input is then checked before the next frame is processed.

The main modules are:

- camera.py – webcam handling
- detector.py – YOLO11 loading and inference
- analytics.py – FPS and object statistics
- renderer.py – drawing boxes, labels and information
- controls.py – keyboard actions
- main.py – command-line arguments and main loop
- __main__.py – allows the program to run with python -m app

9. Workflow

The working process is straightforward:

1. Start the application and read the command-line options.
2. Load the YOLO11n model.
3. Open the webcam.
4. Read a frame.
5. Run YOLO11n inference on the frame.

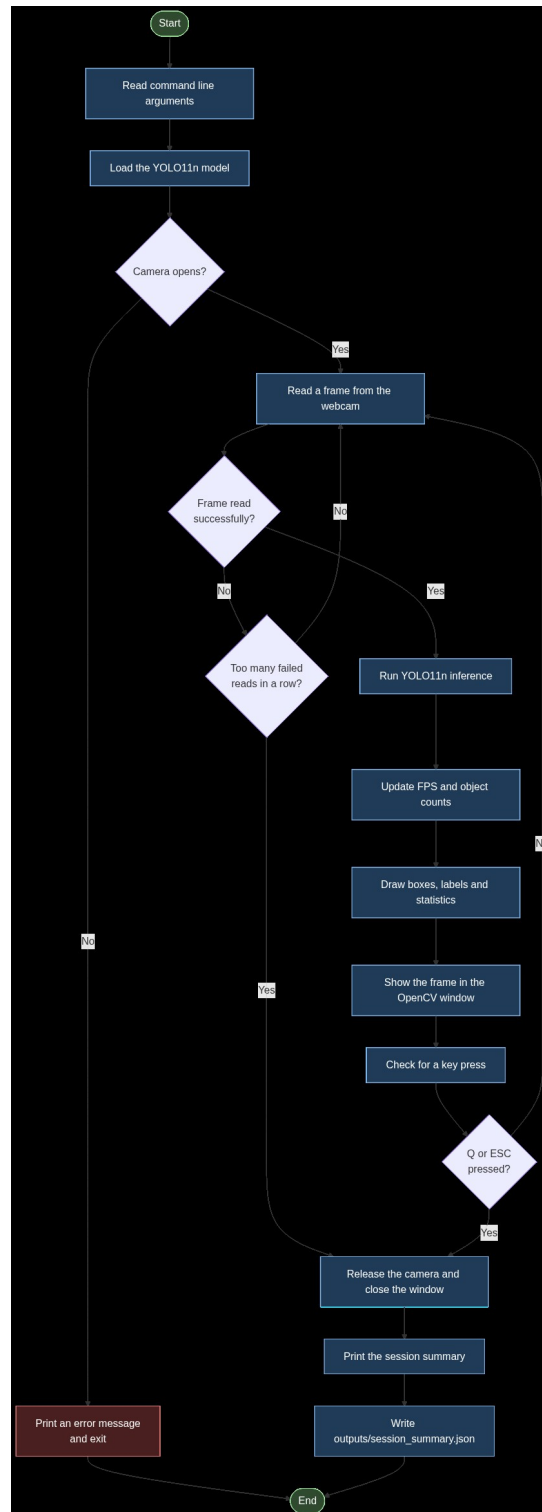


Figure: Workflow diagram.

10.3 Use Case Diagram

The use case diagram shows the actions available to the student/user while using the application.

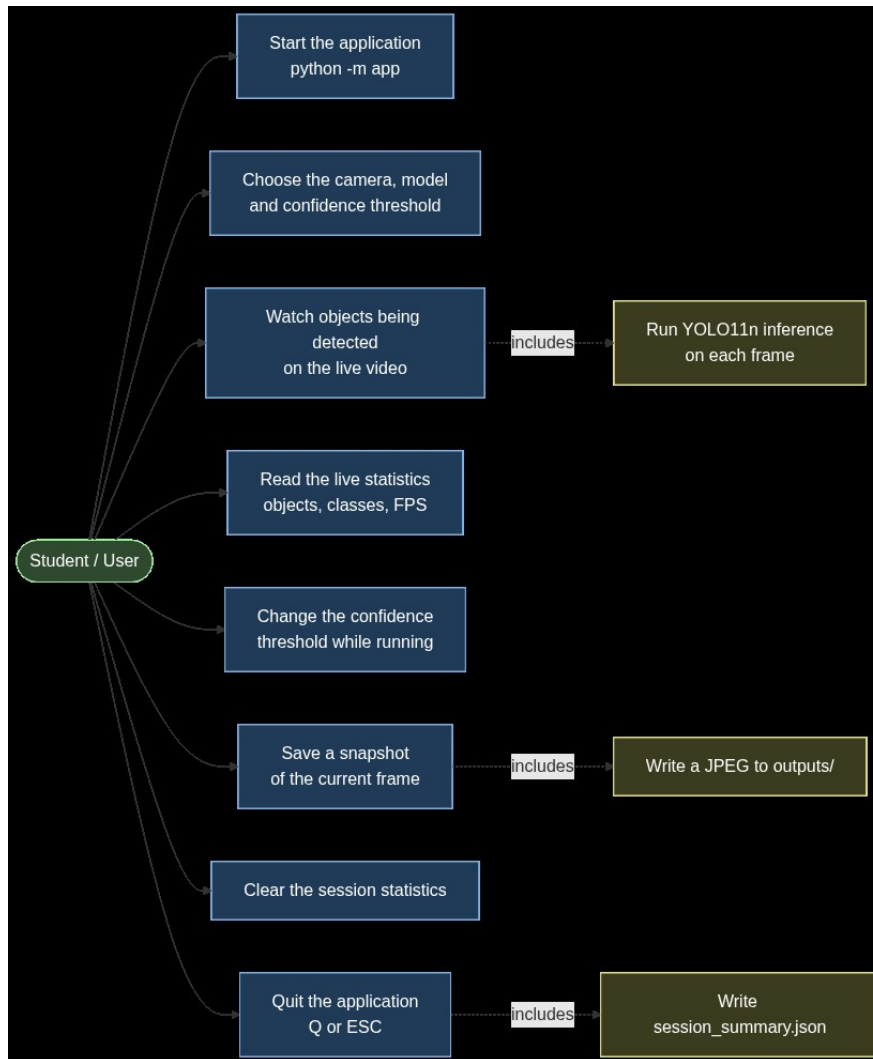


Figure: Use case diagram.

10.4 Sequence Diagram

The sequence diagram shows the order in which the main module interacts with the camera, detector, analytics and renderer.

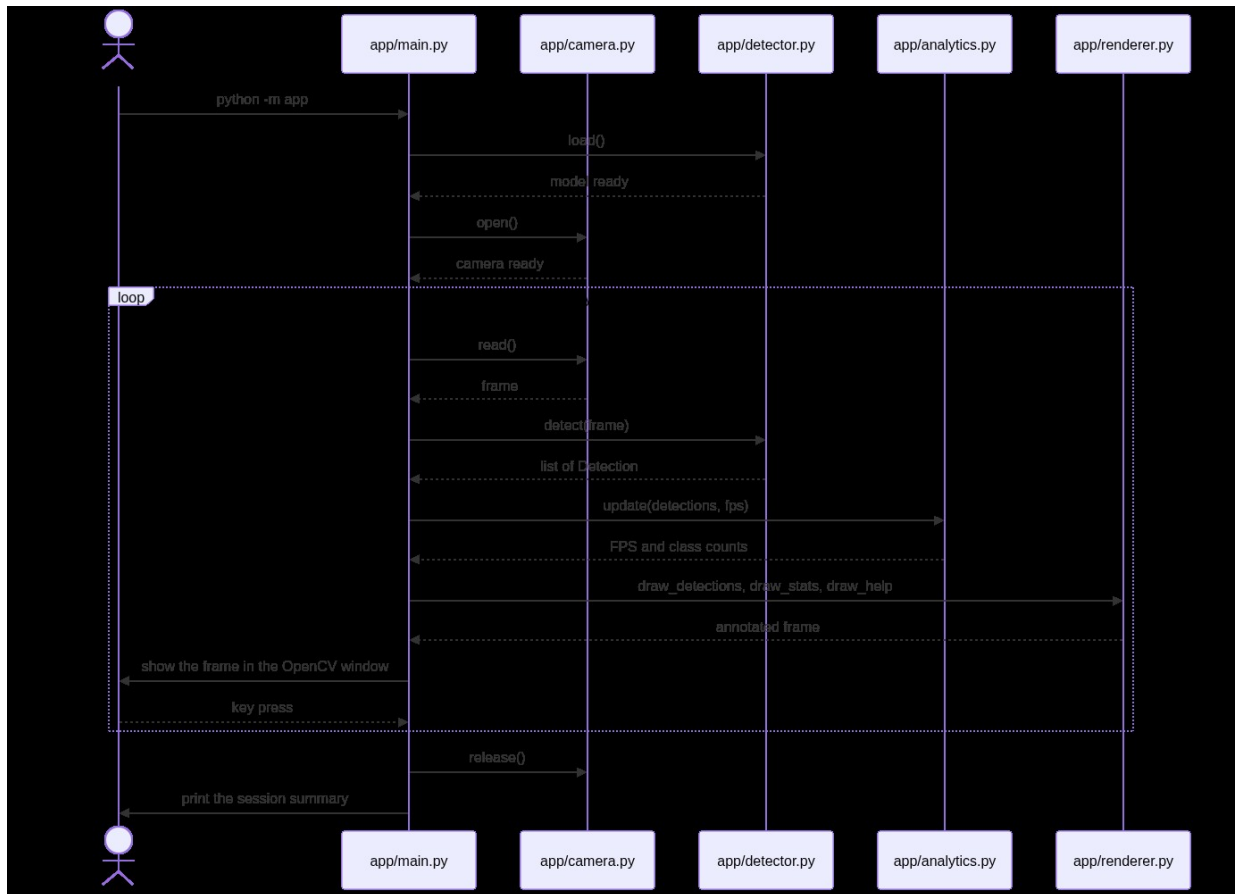


Figure: Sequence diagram.

10.5 Component/Class Diagram

The class diagram gives an overview of the main classes and data structures used by the application.

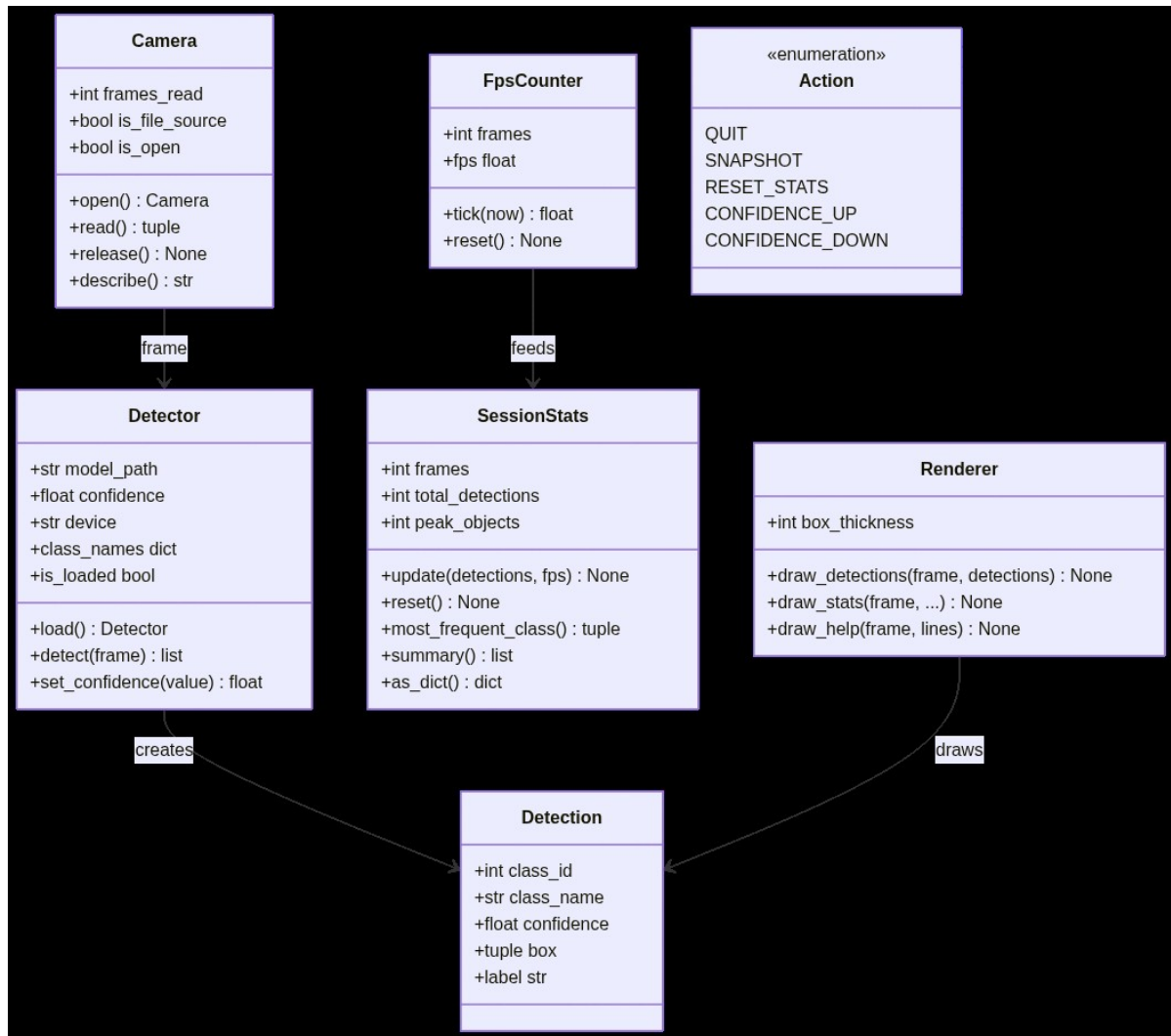


Figure: Component and class diagram.

11. Implementation

11.1 Webcam Capture

OpenCV VideoCapture is used to open the webcam and read frames. The application uses the appropriate capture backend for the operating system when possible. If the camera cannot be opened, a readable error is shown.

11.2 Detection

The detector module loads YOLO11n once and performs inference on each frame. The raw model output is converted into a simple Detection object containing the class ID, class name, confidence and bounding-box coordinates.

11.3 Visualisation

The renderer draws a box around each detection and writes the class name and confidence above the box. It also displays the current FPS, object information and keyboard controls.

11.4 Analytics

The analytics module keeps track of the frame rate, number of detections and class totals. These values are displayed during the run and saved in the final session summary.

11.5 Confidence Threshold

The default confidence threshold is 0.25. The + and – keys change it by 0.05 within the allowed range. A lower threshold can show more weak detections, while a higher threshold keeps fewer, stronger detections.

11.6 Output Files

Pressing S saves the current annotated frame to outputs/. When the program exits, the application writes outputs/session_summary.json with the main session statistics.

12. User Controls

Q / ESC – Quit the application

S – Save a snapshot

C – Clear the session statistics

+ – Increase confidence threshold

– – Decrease confidence threshold

The controls are also displayed in the application window so they are easy to remember while running the program.

13. Testing

A unit-test suite of 62 tests was used to check the main application logic. The tests do not require a webcam or a model download, which makes them quick to run.

Test breakdown:

- test_main.py – 11 tests
- test_camera.py – 10 tests
- test_detector.py – 13 tests
- test_analytics.py – 16 tests
- test_controls.py – 12 tests

Total: 62 tests

The tests cover argument handling, camera behaviour and error handling, detection data, confidence handling, FPS calculation, object counting, session statistics, keyboard mappings and output-file handling.

14. Results

14.1 Test Run

The complete unit-test suite passed successfully with 62 tests passed.

14.2 Webcam Demonstration

The application was also run using a physical webcam. The model loaded successfully and the live window showed object detections and statistics. The S key was used to save an annotated snapshot and the program generated the session summary JSON on exit.

14.3 Recorded Session

The following values are from the actual webcam session used for the report. They describe that run only.

Conditions of the recorded webcam run

Parameter	Value
Model	yolo11n.pt (pretrained YOLO11n)
Object classes	80 (COCO)
Inference device	Apple Silicon GPU (MPS)
Confidence threshold	0.25
Camera	Device index 0, AVFoundation
Capture resolution	1920 × 1080

Recorded results

Measurement	Value
Frames processed	1063
Total detections	1573
Most objects in one frame	5
Average detections/frame	1.48
Average confidence	0.739
Average FPS	16.0
Peak FPS	19.8
Duration	1 minute 14 seconds
Most frequent class	person (1290)

The recorded values are observations from one webcam session. They are useful for showing that the application was running, but they are not a formal accuracy benchmark.

Per-class totals

Class	Count
-------	-------

person	1290
cell phone	85
toothbrush	45
microwave	44
tv	37
bowl	22
dog	14
refrigerator	9
oven	8
chair	7
wine glass	6
bottle	3
fork	2
potted plant	1

15. Screenshots and Evidence

The following screenshots are taken from the actual project run. They show the live detection result, the terminal output and the project design diagrams.



Figure: Live webcam detection snapshot.

```
Loaded yolo11n.pt with 80 object classes.
=====
Model      : yolo11n.pt
Device     : Apple Silicon GPU (MPS)
Confidence : 0.25
Capture    : camera index 0 (AVFoundation)
Output     : /Volumes/T7 Shield/PROJECTS/Object Detection Model/outputs
=====
Q / ESC - Quit
S - Save a snapshot
C - Clear statistics
+ / - - Change confidence threshold
=====

Snapshot saved: outputs/snapshot_2026_09_18_004644.jpg
=====
Session summary
=====
Frames processed      : 1063
Total detections      : 1573
Most objects in one frame: 5
Average detections/frame : 1.48
Average confidence    : 0.739
Average FPS           : 16.0
Peak FPS              : 19.8
Duration              : 0:01:14
Most frequent class   : person (1290)
=====

Session statistics written to: outputs/session_summary.json

abhijairaghuvanshi@abhijais-MacBook-Air Object Detection Model %
[Restored 18 Sep 2026 at 12:27:27 PM]
Last login: Fri Sep 18 00:06:36 on ttys000
abhijairaghuvanshi@abhijais-MacBook-Air Object Detection Model %
```

Figure: Terminal output from the recorded run.

16. Limitations

- The model is limited to the object classes available in its pretrained COCO setup.
- Detection can be affected by lighting, motion blur, small objects and objects that are partly hidden.
- FPS depends on the computer, camera resolution and model size.
- Only one webcam is used at a time.
- Objects are detected independently in each frame; there is no tracking of the same object across frames.
- No labelled dataset was created for this project, so formal precision, recall, F1 or mAP results are not reported.

17. Challenges and Learning Outcomes

17.1 Challenges

Some practical issues came up during development, especially camera access, installing the required packages, loading the model and keeping the live display responsive. Testing the application logic without always depending on the webcam was also useful.

17.2 Learning Outcomes

The project gave practical experience with OpenCV, real-time video processing, YOLO inference, modular Python programming, keyboard-based controls, unit testing and Git/GitHub. It also helped in understanding the difference between a model's confidence value and a formal accuracy measurement.

18. Future Enhancements

Possible improvements include object tracking, custom training on a project-specific dataset, video-file input, line-crossing counts, adaptive inference resolution and a larger labelled test set for proper accuracy evaluation.

19. Conclusion

The project successfully implements a real-time webcam object detection application using YOLO11n. It captures frames, performs detection, displays the results and provides simple live statistics and controls. The application is split into separate modules and the main logic is covered by 62 unit tests. The physical webcam run also confirmed that the complete pipeline works together and that snapshots and session summaries are generated correctly.

20. References

1. Ultralytics, "YOLO11 – Ultralytics Documentation."
2. Ultralytics, "Predict Mode – Ultralytics Documentation."
3. Ultralytics, Ultralytics YOLO11 software repository.
4. OpenCV Documentation.
5. Redmon, J., Divvala, S., Girshick, R. and Farhadi, A. (2016), "You Only Look Once: Unified, Real-Time Object Detection," CVPR.
6. Lin, T.-Y. et al. (2014), "Microsoft COCO: Common Objects in Context," ECCV.
7. Project GitHub repository: github.com/Abhijai10/real-time-object-detection-yolo11